The Problem of Consensus in Distributed Systems

Distributed Systems / Case Study

Andrea Omicini
andrea.omicini@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
Alma Mater Studiorum – Università di Bologna

Academic Year 2025/2026

Next in Line...

Agreement Problems I

The agreement problem in general

- when the *different components* of a system have potentially *divergent views* over the system state, and about what is happening in the system
- a coherent overall system behaviour can be achieved only when all the components come to agree about everything *relevant* for the system itself—state, events, ...
- the key point being that they reach the very *same* conclusion, independently of *what* that is about

Agreement Problems II

Examples of agreement problems[Fischer, 1983]

- *data managers* in a distributed database system need to agree on whether to *commit* or *abort* a given (distributed) transaction
- in a *replicated file system*, the nodes might need to *agree on where* the file copies are supposed to reside
- in a flight control system for airplane, the engine control module and the flight surface control module need to *agree* on whether to *continue* or *abort* a landing in progress

Some Basic Ontology I

Components & connectors

- non-trivial computational systems are *concurrent* or, more generally, *distributed systems*
  - that is, respectively, systems where *more than one computation* occur either *logically* or *physically* at the same time
- correspondingly, concurrent / distributed systems are typically modelled as collections of computing (logical) *processes* or (physical) devices (*processors*)
- interconnected somehow so as to interact
- e.g.,
  - via one-to-one direct communication channels (*message-passing architectures*)
  - via shared communication channels (*shared-memory architectures*)

Some Basic Ontology II

Concurrent or distributed? Processors or processes?

- and, what about parallel?
- meanwhile, we just accept that the literature uses the terms more or less *interchangeably*, whenever no harm is done
- whenever we focus on those systems that are *unaffected by the physical nature* of the problem to be solved, the distinction is not relevant

Some Basic Ontology III

Time model

synchronous where there is

- a bound Δ on the transmission delay of messages, and
- a bound Φ on the relative speed of processes

which allows accurate *failure detection* of processes

asynchronous otherwise

- which typically models a system with unpredictable load on the network and on the CPUs

where, as a consequence, it is never possible to know whether a process has crashed or not

Some Basic Ontology IV

Failure model

- either logical processes or physical processors may *fail*, and in diverse ways
- communication links may fail, and in diverse ways
- *fault tolerance* is the general property of distributed systems to keep working beyond faults—of any sorts

Agreement & Fault Tolerance

Agreement problem in computational systems[Fischer, 1983]

- *agreement problems* involve a *system of processes*, some of which may be *faulty*
- a fundamental problem of *fault-tolerant* distributed computing is for the *reliable* processes to reach a consensus

Consensus

What is *consensus*?

The process by which we reach *agreement* over system state between unreliable machines connected by (possibly) asynchronous networks

Why consensus?

- when processes fail, the reliable processes need to agree on
  - what has happened till now in the system, and
  - what is going to happen from now on

  so that current system state & evolution are both consistent
- e.g., for system recovery

Consensus & Fault Tolerance

- consensus is particularly important to provide a distributed system of some level of fault tolerance because it is essential to ensure the consistency of replicas
- when a crash makes it impossible to ensure replica synchronisation, a recovery process involves the consensus among the many replicas
  - about the state of each replica
  - and the events handled / to handle

Many Consensus Problems I

Basic consensus[Fischer, 1983]

- most basic form: achieving consensus on a *single bit*
  - a fixed number n of processors, possibly *faulty*
  - each processor has an initial bit x_i
  - the single-bit consensus problem is for the non-faulty processes to agree on a bit y, called the *consensus value*
- *solvability* depends on the existence of a protocol such that all non-faulty processors terminate with the same value y
- different requirements lead to different problems
  - e.g. *dependency requirements*—y should depend on x somehow

    non-triviality $\forall y \in \{0,1\}$, some vector $\overline{x_i}$ and protocol run lead to y

    strong unanimity if $\forall_i x_i = x \in \{0,1\}$ then $y = x$

    weak unanimity same as strong if no run-time failures occur

Many Consensus Problems II

Interactive consistency problem[Fischer, 1983]

- analogous to the (single-bit) consensus problem
- except that the goal of the protocol is for the non-faulty processes to agree on a vector y, called the *consensus vector*
- dependency requirements

  strong $\forall_i y_i = x_i$ if i is not faulty

  weak $\forall_i y_i = x_i$ if i is not faulty, provided that no failures actually occur during the execution of the protocol
- basically, consensus is about what any processor thinks about the initial value

Many Consensus Problems III

Generals problem[Fischer, 1983]

- one specific processor (the *general*) tries at sending its initial bit x to all the others
- all the reliable processes have to reach consensus on the bit y, with the dependency requirements

  strong $y = x$ if the general is not faulty

  weak $y = x$ if no failures occur during the execution of the protocol
- the problem is also known as the *reliable broadcast* problem
  - where the general is known as the *transmitter*

Example: Transaction Commit Problem

Transaction commit in distributed databases[Dolev and Strong, 1982]

- the problem is for all the data manager processes that have participated in the processing of a particular transaction to agree on whether to install the transaction's results in the database or to discard them
- the latter action might be necessary, for example, if some data managers were, for any reason, unable to carry out the required transaction processing
- whatever decision is made, *all data managers must make the same decision* in order to preserve the consistency of the database

So What?

Abstract representation of problems & solutions

- many technical and real-world issues can be reduced to that sort of problem—or, to some version of its
- an abstract, general, synthetic way to represent problems and provide general solutions
- . . . how hard can this be?

What is the Problem?

Faults[Fich and Ruppert, 2003]

- reaching the type of agreement needed for the commit problem is straightforward if the participating *processes* and the *network* are completely *reliable*
- however, real systems are subject to a number of possible *faults*, such as process crashes, network partitioning, and lost, distorted, or duplicated messages
  - possibly including *Byzantine* types of failure—where faulty processes might go completely out of control, or behave malevolently
- one would therefore look for an agreement protocol that is *as reliable as possible in the presence of such faults*
- yet, any protocol can be overwhelmed by too frequent or severe faults
- so the best that one can hope for is a protocol that is tolerant to a prescribed number of "expected" faults.

Next in Line...

Impossibility, Again I

"Hundreds of impossibility results for distributed computing"

"We survey results from distributed computing that show tasks to be impossible, either outright or within given resource bounds, in various models. The parameters of the models considered include synchrony, fault-tolerance, different communication media, and randomization. The resource bounds refer to time, space and message complexity. These results are useful in understanding the inherent difficulty of individual problems and in studying the power of different models of distributed computing. There is a strong emphasis in our presentation on explaining the wide variety of techniques that are used to obtain the results described."

[Fich and Ruppert, 2003]

Impossibility, Again II

Not just CAP, then

- distributed systems are easy to conceive and build
- as well as easy to crash, difficult to understand and debug
- formal proof are fundamental for serious engineering
- yet, properties are difficult to prove
- so, knowing how the models affect the properties of the systems is essential
- as well as knowing which problems we can *not* solve

FLP (Fischer-Lynch-Paterson)

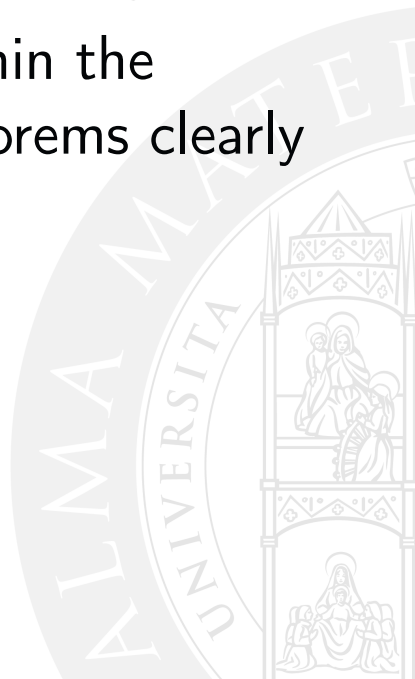
"Impossibility of distributed consensus with one faulty process"

- in a fully-asynchronous system, *no consensus protocol can tolerate even one single crash failures* under the mere requirement of non-triviality
- an impossibility result called FLP after the initials of the authors[Fischer et al., 1985]
- so, even the simplest case leads to an impossibility result
  - no Byzantine failures considered
  - the message system is assumed to be reliable—that is, it delivers all messages correctly and exactly once
  - yet, the stopping of a single process at an inopportune time can cause any distributed commit protocol to fail to reach agreement

→ that simple version of the consensus problem *has no robust solution* without further assumptions about the computing environment or still greater restrictions on the kind of failures to be tolerated

So What?

- again, are we going to *give up* on so many (relevant, frequently occurring) *problems*?
- or, to give up on some essential *features* of the system solving them?
- or, instead, are we going to elaborate some *solution* within the well-defined *boundaries* that (as usual) impossibility theorems clearly set for us?



Next in Line...

Facing Harsh Reality with Purpose I

"Distributed Consensus: Making Impossible Possible" [Howard, 2016]

In this talk, we explore how to construct resilient distributed systems on top of unreliable components. Starting, almost two decades ago, with Leslie Lamport's work on organising parliament for a Greek island. We will take a journey to today's datacenters and the systems powering companies like Google, Amazon and Microsoft. Along the way, we will face interesting impossibility results, machines acting maliciously and the complexity of today's networks. Ultimately, we will discover how to reach agreement between many parties and from this, how to construct new fault-tolerance systems that we can depend upon everyday.

Facing Harsh Reality with Purpose II

Solution to FLP

- in practice we accept that sometimes the system will not be available
  - we mitigate this using *timers* and *backoffs*
- in theory we make weaker assumptions about the synchrony of the system
  - e.g., messages arrive within a year

Paxos[Lamport, 1998] I

Paxos consensus algorithm for reaching agreement on a single value

- first presented in 1989 as a technical report by Lamport, then recasted in a more academically-viable form[Lamport, 1998]
- generalised with respect to State Machine Replication (SMR)[Lampson, 1996]
- formally described and proven correct[De Prisco et al., 1997]
- nicely explained[Lamport, 2001]

Paxos[Lamport, 1998] II

Paxos features

- Paxos is an algorithm for implementing *fault-tolerant consensus*
- it runs on a completely connected network of n processes and tolerates up to f failures, where $n \geq 2f + 1$
- in Paxos, processes can crash and messages may be lost, but byzantine failures are ruled out
- Paxos primarily guarantees agreement and *validity*
  - termination is ensured only if there is a sufficiently long interval during which no process restarts the protocol

Paxos[Lamport, 1998] III

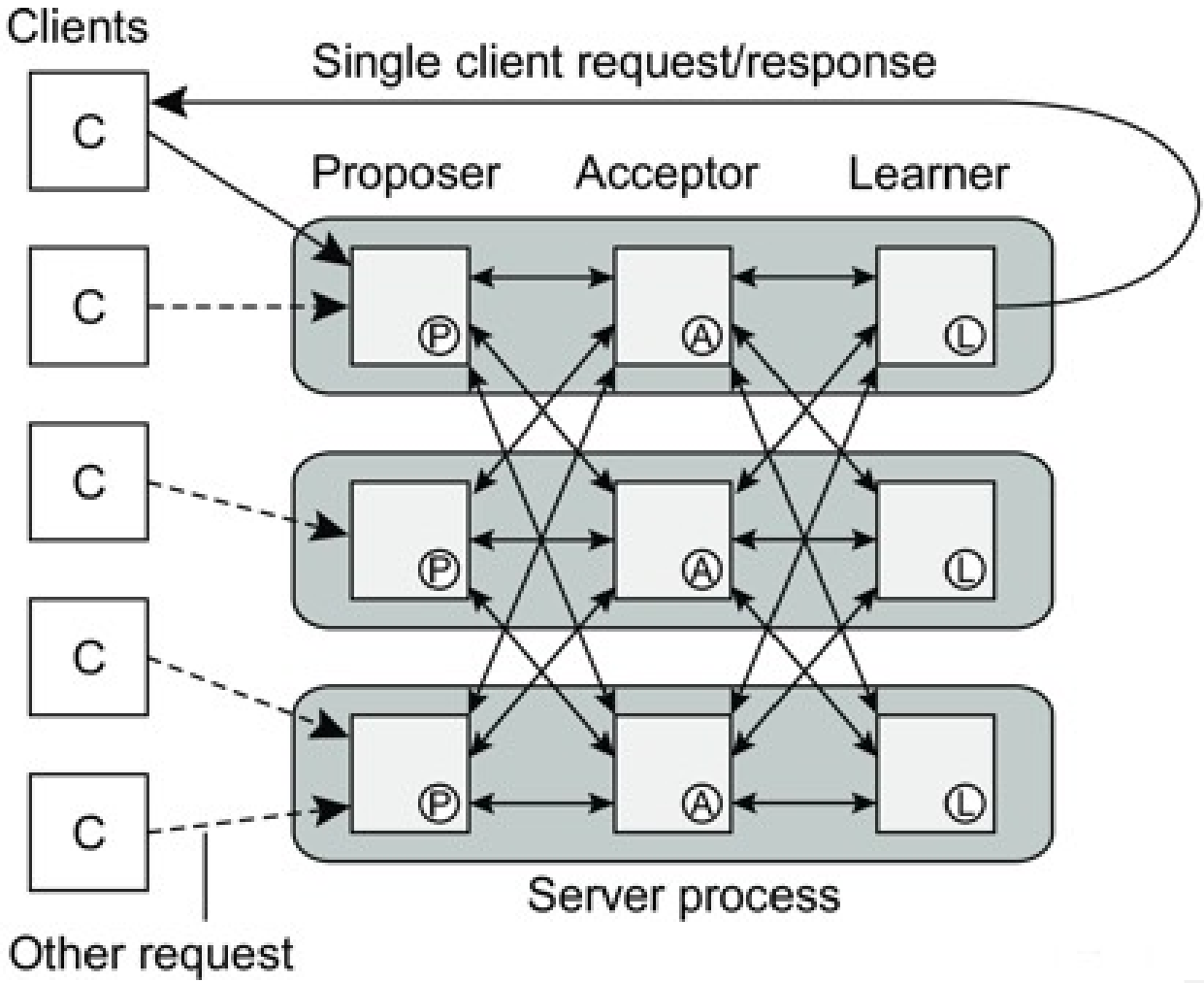
Process roles

A process may play three different roles

- proposer
  - proposers submit proposed values on behalf of clients

  each role with a different responsibility
- acceptor
  - acceptors decide the candidate values for the final decision
- learner
  - learners collect these information from the acceptors and report the final decision back to the clients

Paxos[Lamport, 1998] IV



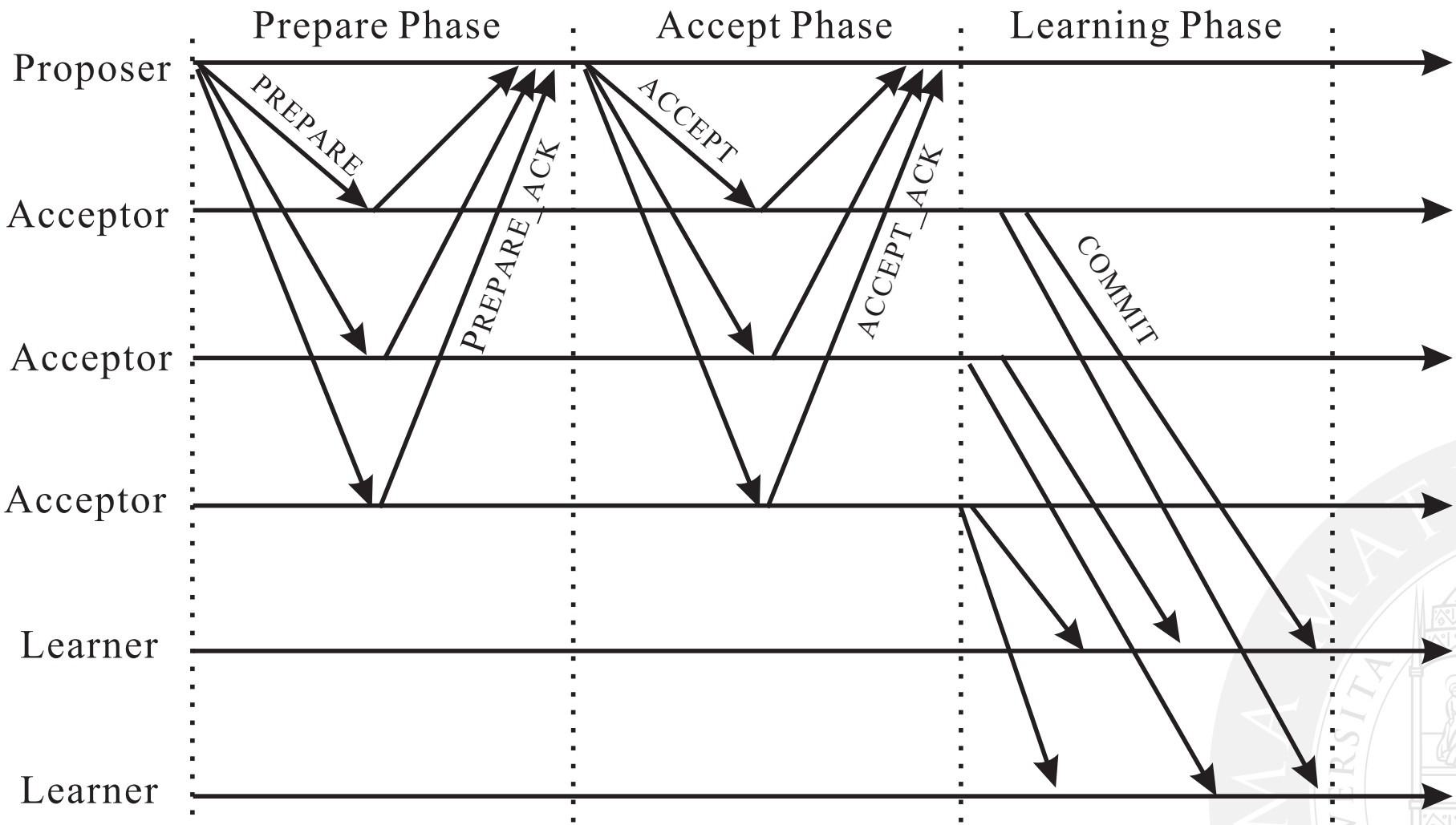
Logical process view of Paxos[Tanenbaum and van Steen, 2017]

Paxos[Lamport, 1998] V

Basic idea

- two-phase process
  - *promise*
  - *commit*
- majority agreement
- monotonically increasing numbers
  - to label proposals

Paxos[Lamport, 1998] VI



Basic Paxos operation[Zhao, 2014]

Paxos Made Simple[Lamport, 2001]

Or, even simpler

- values are proposed (*proposer*)
- no single acceptor of proposals: multiple (*odd* number) *acceptors*
  - *quorum*: if the bare majority of acceptors, then it is chosen
- once a value has been chosen, future proposals must propose the same value
- this means a *two-phase protocol* would be needed
  - in the first phase (*prepare*) need to figure out if any value has already been chosen
  - and propose accordingly in the second phase (*accept*)
- which may either lead to a value actually chosen, or, to a iteration of the phases...
  - which theoretically might lead to *livelock*

Paxos Phases[Ghosh, 2014] I

- a proposal sent by a proposer is a pair (v, n) where v is a value and n is a sequence number
- to deal with possible crashes, *multiple acceptors* are selected for the proposal
- proposal must be endorsed by at least one acceptor before it becomes a candidate for the final decision
- the sequence number is used to distinguish between successive attempts to invoke the protocol
- upon receiving a proposal with a larger sequence number from a given process, acceptors discard the proposals with lower sequence numbers
- eventually, an acceptor accepts the majority's choice

Paxos Phases[Ghosh, 2014] II

Phase 1: Preparatory phase

1.1 each proposer sends a proposal (v, n) to each acceptor

1.2 if n is the largest sequence number of a proposal received by an acceptor, then it sends an $ack(n, \perp, \perp)$ to its proposer

- which is a promise that it will ignore all proposals numbered lowered than n

- however, in case an acceptor has accepted a proposal with a sequence number $n' < n$ and a proposed value v, it responds with $ack(n, v, n')$
  - which implies that the proposer has no point in trying to submit another proposal with a larger sequence number

Paxos Phases[Ghosh, 2014] III

Phase 2: Request for acceptance of an input value

2.1 if a proposer receives $ack(n, \perp, \perp)$ from a majority of acceptors, then it sends $accept(v, n)$ to all acceptors, asking them to accept this value

– if however, an acceptor returns an $ack(n, v, n')$ to the proposer in phase 1

- meaning that it already accepted proposal with value v

then the proposer must include the value v with the highest sequence number in its request to the acceptors

2.2 an acceptor accepts a proposal (v, n) unless it has already promised to consider proposals with a sequence number $n' > n$

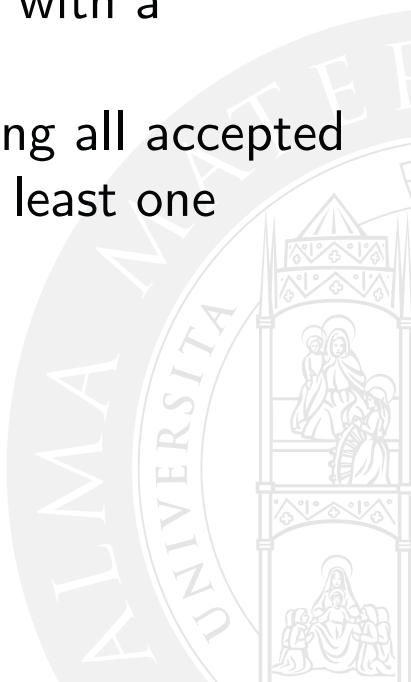
Paxos Phases[Ghosh, 2014] IV

Phase 3: Final decision

- when a majority of the acceptors accept a proposed value, it becomes the final decision value
- the acceptors multicast the accepted value to the learners, which enables them to determine if a proposal has been accepted by a majority of acceptors
- the learners convey it to the client processes invoking the consensus

Paxos: Observations[Ghosh, 2014]

- an acceptor accepts a proposal with a sequence number n if it has not sent a promise to any proposal with a sequence number $n' > n$
- if a proposer sends an $accept(v, n)$ message in phase 2, then
  - either no acceptor in a majority has accepted a proposal with a sequence number $n' < n$
  - or, v is the value in the highest numbered proposal among all accepted proposals with sequence numbers $n' < n$ accepted by at least one acceptor in a majority



State Machine Replication[Schneider, 1990] I

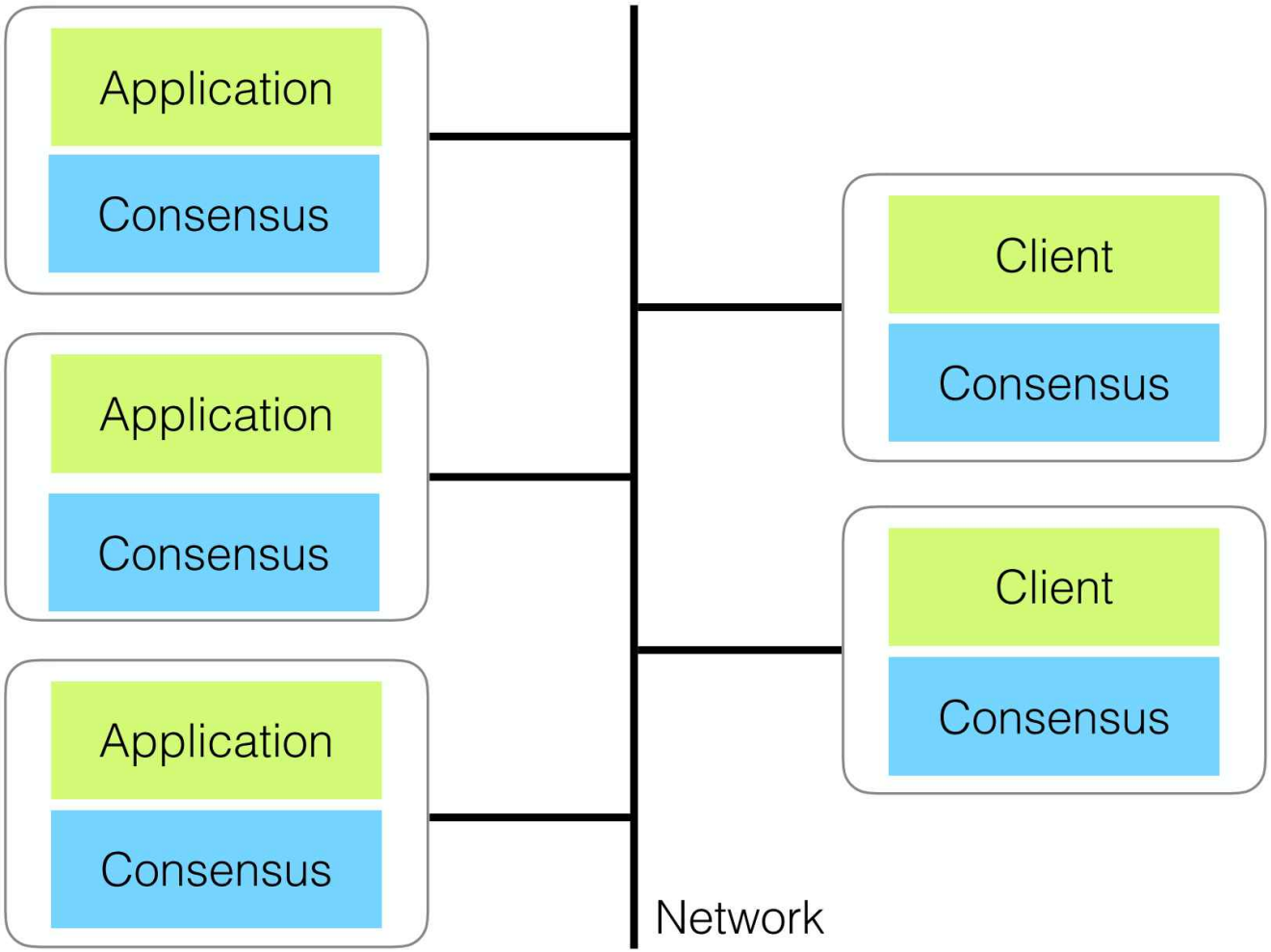
- *replication* relates to consensus in that all (distributed) replicas of a resource need to agree on the same state over time
- on the other hand, the *state machine* approach is a general method for implementing fault-tolerant services in distributed systems

Basic idea

- the same state machine is replicated over a distributed system
- each one has a *consensus module*
- any operation over any replicated machine is performed only if and when *all* machines have agreed over the same ordering of execution

- shift from *consensus over distributed state* to *consensus over distributed actions* over state
- assumption of *determinism*

State Machine Replication[Schneider, 1990] II



[Howard, 2016]

Beyond Paxos I

Multi-Paxos[Van Renesse and Altinbuken, 2015]

- SMR needs consensus over an array of values, Paxos just deals with a single value
- Multi-Paxos basically leverage on one separate Paxos algorithm (indexed) on each entry of the array
- plus *leader election* to reduce proposer conflict (so presumably dropping off iterations)
- not really well-specified overall

Beyond Paxos II

Other Paxos-based algorithms

- Fast Paxos[Lamport, 2006]
- ZooKeeper[Hunt et al., 2010]
  - ZAB (ZooKeeper atomic broadcast)[Reed and Junqueira, 2008]
- Egalitarian Paxos[Moraru et al., 2013]
- Raft Consensus[Ongaro and Ousterhout, 2014]

An Example: Chubby by Google

How Google Uses Paxos[Chandra et al., 2007]

- fault-tolerant database using the Paxos consensus algorithm
- Google's Chubby[Burrows, 2006]
  - used extensively inside Google systems
  - e.g. Google File System, Cloud Bigtable, ...
  - as a reliable lock service
- an informal introduction to Chubby can be found here

  https://medium.com/coinmonks/chubby-a-centralized-lock-service-for-distributed-applications-390571273052

Next in Line...

Some Lessons Learnt I

General lessons

- impossibility results define the space of the problems we can actually solve
- reasoning about the relationship between the formal models of the problems and their practical instances typically provide us with some glimpse for some technically-feasible solution
- formal models are essential to a sound engineering of distributed systems
  - and, not just within very complex settings

Some Lessons Learnt II

Specific lessons

- *faults* are at the same time the sources of the problems or distributed systems, and the reason we need distributed systems
- *consensus* is one of the most expressive and general way to model relevant application problems in a distributed setting
- on the one hand, the complexity of consensus both defies our common sense and causes archetypal cases of impossibility
- on the other hand, protocols can be defined that actually solve the problem in meaningful cases with some relaxation of basic hypotheses
  - e.g., Paxos & friends

The Problem of Consensus in Distributed Systems

Distributed Systems / Case Study

Andrea Omicini
andrea.omicini@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
Alma Mater Studiorum – Università di Bologna

Academic Year 2025/2026

References I

[Burrows, 2006] Burrows, M. (2006).
The Chubby lock service for loosely-coupled distributed systems.
In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2006)*, pages 335–350, Berkeley, CA, USA. USENIX Association
https://dl.acm.org/doi/10.5555/1298455.1298487

[Chandra et al., 2007] Chandra, T. D., Griesemer, R., and Redstone, J. (2007).
Paxos made live: An engineering perspective.
In *26th Annual ACM Symposium on Principles of Distributed Computing (PODC 2007)*, pages 398—407, New York, NY, USA. ACM
DOI:10.1145/1281100.1281103

[De Prisco et al., 1997] De Prisco, R., Lampson, B. W., and Lynch, N. A. (1997).
Revisiting the Paxos algorithm.
In Mavronicolas, M. and Tsigas, P., editors, *Distributed Algorithms*, volume 1320 of *Lecture Notes in Computer Science*, pages 111–125. Springer-Verlag, Saarbruken, Germany
DOI:10.1007/BFb0030679

[Dolev and Strong, 1982] Dolev, D. and Strong, H. R. (1982).
Distributed commit with bounded waiting.
In *IEEE Symposium on Reliability in Distributed Software and Database Systems (SRDS 1982)*, pages 53–60. IEEE Computer Society
(APICe)

[Fich and Ruppert, 2003] Fich, F. and Ruppert, E. (2003).
Hundreds of impossibility results for distributed computing.
Distributed Computing, 16(2-3):121–163
DOI:10.1007/s00446-003-0091-y

References II

[Fischer, 1983] Fischer, M. J. (1983).
The consensus problem in unreliable distributed systems (a brief survey).
In Karpinski, M., editor, *Foundations of Computation Theory*, pages 127–140, Berlin, Heidelberg. Springer Berlin Heidelberg
DOI:10.1007/3-540-12689-9_99

[Fischer et al., 1985] Fischer, M. J., Lynch, N. A., and Paterson, M. S. (1985).
Impossibility of distributed consensus with one faulty process.
Journal of the ACM (JACM), 32(2):374–382
DOI:10.1145/3149.214121

[Ghosh, 2014] Ghosh, S. (2014).
Distributed Systems: An Algorithmic Approach.
Chapman & Hall/CRC Computer and Information Science Series. CRC Press, 2nd edition
DOI:10.1201/b17224

[Howard, 2016] Howard, H. (2016).
Distributed consensus: Making impossible possible'
https://www.slideshare.net/InfoQ/distributed-consensus-making-the-impossible-possible

[Hunt et al., 2010] Hunt, P., Konar, M., Junqueira, F. P., and Reed, B. (2010).
{ZooKeeper}: Wait-free coordination for Internet-scale systems.
In *2010 USENIX Annual Technical Conference (USENIX ATC 10)*
https://www.usenix.org/legacy/event/atc10/tech/full_papers/Hunt.pdf

References III

[Lamport, 1998] Lamport, L. (1998).
The part-time parliament.
ACM Transactions on Computer Systems, 16(2):133—169
DOI:10.1145/279227.279229

[Lamport, 2001] Lamport, L. (2001).
Paxos made simple.
ACM SIGACT News, 32(4):51–58.
Distributed Computing Column 5, by Sergio Rajsbaum
DOI:10.1145/279227.279229

[Lamport, 2006] Lamport, L. (2006).
Fast Paxos.
Distributed Computing, 19(2):79–103
DOI:10.1007/s00446-006-0005-x

[Lampson, 1996] Lampson, B. W. (1996).
How to build a highly available system using consensus.
In Babaoğlu, Ö. and Marzullo, K., editors, *Distributed Algorithms*, volume 1151 of *Lecture Notes in Computer Science*, pages 1–17. Springer-Verlag, Berlin, Germany
DOI:10.1007/3-540-61769-8_1

[Moraru et al., 2013] Moraru, I., Andersen, D. G., and Kaminsky, M. (2013).
There is more consensus in egalitarian parliaments.
In *24th ACM Symposium on Operating Systems Principles (SOSP'13)*, pages 358–372, New York, NY, USA. Association for Computing Machinery
DOI:10.1145/2517349.2517350

References IV

[Ongaro and Ousterhout, 2014] Ongaro, D. and Ousterhout, J. K. (2014).
In search of an understandable consensus algorithm.
In Gibson, G. and Zeldovich, N., editors, *2014 USENIX Annual Technical Conference, USENIX ATC '14, Philadelphia, PA, USA, June 19-20, 2014*, pages 305–319. USENIX Association
https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

[Reed and Junqueira, 2008] Reed, B. and Junqueira, F. P. (2008).
A simple totally ordered broadcast protocol.
In *2nd Workshop on Large-Scale Distributed Systems and Middleware (LADIS'08)*, New York, NY, USA. Association for Computing Machinery
DOI:10.1145/1529974.1529978

[Schneider, 1990] Schneider, F. B. (1990).
Implementing fault-tolerant services using the state machine approach: A tutorial.
ACM Computing Surveys, 22(4):299–319
DOI:10.1145/98163.98167

[Tanenbaum and van Steen, 2017] Tanenbaum, A. S. and van Steen, M. (2017).
Distributed Systems. Principles and Paradigms.
Pearson Prentice Hall, 3rd edition
https://www.distributed-systems.net/index.php/books/ds3/

[Van Renesse and Altinbuken, 2015] Van Renesse, R. and Altinbuken, D. (2015).
Paxos made moderately complex.
ACM Computing Surveys, 47(3)
DOI:10.1145/2673577

References V

[Zhao, 2014] Zhao, W. (2014).
Building Dependable Distributed Systems.
Wiley
DOI:10.1002/9781118912744